



DSA

Module 9

Sorting

Author
Srinivas Dande



My **JAVA**
LEARNING
CENTER



9. Sorting

- ♦ Sorting Algorithm is used to arrange elements of an array/list in an order specified.
- ♦ Following are the list of Sorting Algorithms

- 1) Bubble Sort
- 2) Selection Sort
- 3) Insertion Sort
- 4) Merge Sort
- 5) Quicksort
- 6) Heap Sort
- 7) Counting Sort
- 8) Radix Sort
- 9) Bucket Sort
- 10) Shell Sort



9.1. Complexity of Sorting Algorithms

<u>Sorting Algorithm</u>	<u>Time Complexity Best</u>	<u>Time Complexity Worst</u>	<u>Time Complexity Average</u>	<u>Space Complexity</u>
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Max
Radix Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Max
Bucket Sort	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$
Shell Sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(1)$



9.2. Stability of Sorting Algorithms

- a) Stable Sorting Algorithms
- b) Unstable Sorting Algorithms

a) Stable Sorting Algorithms:

- ♦ If equal elements are not reordered as a result of sorting then that sorting algorithm is Stable.

Ex:

Bubble Sort
Insertion Sort
Merge Sort
Counting Sort
Radix Sort
Bucket Sort

b) Unstable Sorting Algorithms:

- ♦ If equal elements are reordered as a result of sorting then that sorting algorithm is Unstable.

Ex:

Selection Sort
Quicksort
Heap Sort
Shell Sort



9.3. Sorting in Java

- a) Arrays.sort()
- b) Collections.sort()

a) Arrays.sort():

- ♦ Arrays.sort() can be used in two ways.
 - 1) For Array of Primitives(int,char,double etc)
 - 2) For Array of Objects(Integer, Student, Customer etc)

1) For Array of Primitives(int,char,double etc):

- ♦ Sorts the specified array into ascending numerical order.
- ♦ Sorting algorithm is a Dual-Pivot Quicksort
- ♦ This algorithm offers $O(n \log(n))$ performance on many data sets that cause other quicksorts to degrade to $O(n^2)$ performance

2) For Array of Objects(Integer, Student, Customer etc)

- ♦ Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
- ♦ All elements in the array must implement the Comparable interface.
- ♦ This sort is guaranteed to be stable: equal elements will not be reordered as a result of the sort.
- ♦ Sorting algorithm is a MergeSort
- ♦ This algorithm offers $O(n \log(n))$ performance

- ✓ **Arrays.sort() uses QuickSort for Array of Primitives**
- ✓ **Arrays.sort() uses MergeSort for Array of Objects**



b) Collections.sort()

1) For Lists (ArrayList, LinkedList etc)

- ♦ Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
 - ♦ All elements in the array must implement the Comparable interface.
 - ♦ This sort is guaranteed to be stable: equal elements will not be reordered as a result of the sort.
 - ♦ Sorting algorithm is a MergeSort
 - ♦ This algorithm offers $O(n \log(n))$ performance
- ✓ **Collections.sort() uses MergeSort for Collection of Objects**



9.4. Labs on Arrays.sort()

9.4.1. Sort the Array

Lab1.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {

    public static void main(String[] args) {

        int arr1[] = {10,5,20,15,30,25};
        char arr2[] = {'C','D','E','B','A'};

        Arrays.sort(arr1);
        System.out.println(Arrays.toString(arr1));

        Arrays.sort(arr2);
        System.out.println(Arrays.toString(arr2));

    }
}
```



9.4.2. Sort the Sub-Array

Lab2.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {

    public static void main(String[] args) {

        int arr1[] = {10,5,20,15,30,25};
        char arr2[] = {'C','D','E','B','A'};

        Arrays.sort(arr1,2,6);
        System.out.println(Arrays.toString(arr1));

        Arrays.sort(arr2,3,5);
        System.out.println(Arrays.toString(arr2));

    }
}
```



9.4.3. Sort Array in ASC and DESC Order

Lab3.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Collections;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {

    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,Collections.reverseOrder());
        System.out.println(Arrays.toString(arr));

    }
}
```



9.4.4. Sort Array in ASC and DESC Order

Lab4.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyIntegerComparator implements Comparator<Integer>{

    @Override
    public int compare(Integer a, Integer b) {
        return b-a;
    }
}

public class Lab4{

    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr);
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,new MyIntegerComparator());
        System.out.println(Arrays.toString(arr));

    }
}
```



9.4.5. Arrange as Odd First and Even Next

Lab5.java

```
package com.jlcindia.arrays.sort;
import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyEvenComparator implements Comparator<Integer>{
    @Override
    public int compare(Integer a, Integer b) {
        return a%2-b%2;
    }
}

class MyOddComparator implements Comparator<Integer>{
    @Override
    public int compare(Integer a, Integer b) {
        return b%2-a%2;
    }
}

public class Lab5{
    public static void main(String[] args) {

        Integer arr[] = {10,5,20,15,30,25};

        Arrays.sort(arr,new MyEvenComparator());
        System.out.println(Arrays.toString(arr));

        Arrays.sort(arr,new MyOddComparator());
        System.out.println(Arrays.toString(arr));
    }
}
```



9.4.6. Using Comparable

Lab6.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Customer implements Comparable<Customer>{

    int cid;
    String cname;
    long phone;

    public Customer() {}

    public Customer(int cid, String cname, long phone) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + phone + "]";
    }

    @Override
    public int compareTo(Customer cust) {
        return this.cid-cust.cid;
    }
}
```



```
public class Lab6{  
  
    public static void main(String[] args) {  
  
        Customer cust1 = new Customer(102,"sd",222);  
        Customer cust2 = new Customer(103,"ds",333);  
        Customer cust3 = new Customer(101,"sri",111);  
  
        Customer customers[] = {cust1,cust2,cust3};  
  
        Arrays.sort(customers);  
  
        for(Customer cust:customers) {  
            System.out.println(cust);  
        }  
  
    }  
}
```



9.4.7. Using Comparator

Lab7.java

```
package com.jlcindia.arrays.sort;

import java.util.Arrays;
import java.util.Comparator;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyMarksComparator implements Comparator<Student>{

    @Override
    public int compare(Student stu1, Student stu2) {
        return (int) (stu1.marks-stu2.marks);
    }
}

class Student implements Comparable<Student>{

    int sid;
    String sname;
    double marks;

    public Student() {}

    public Student(int sid, String sname, double marks) {
        super();
        this.sid = sid;
        this.sname = sname;
        this.marks = marks;
    }
}
```



```
@Override
public String toString() {
    return "[" + sid + ", " + sname + ", " + marks + "]";
}

@Override
public int compareTo(Student stu) {
    return this.sid-stu.sid;
}
}
public class Lab7{

    public static void main(String[] args) {

        Student stu1 = new Student(102,"sd",70);
        Student stu2 = new Student(103,"ds",60);
        Student stu3 = new Student(101,"sri",50);

        Student students[] = {stu1,stu2,stu3};

        Arrays.sort(students);

        for(Student stu:students) {
            System.out.println(stu);
        }

        Arrays.sort(students,new MyMarksComparator());

        for(Student stu:students) {
            System.out.println(stu);
        }

    }
}
```



9.5. Labs on Collections.sort()

9.5.1. Sort the List in ASC and DESC

Lab8.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab8 {
    public static void main(String[] args) {

        List<Integer> mylist = new ArrayList<>();
        mylist.add(10);
        mylist.add(5);
        mylist.add(20);
        mylist.add(25);
        mylist.add(15);

        Collections.sort(mylist);
        System.out.println(mylist);

        Collections.sort(mylist,Collections.reverseOrder());
        System.out.println(mylist);

    }
}
```



9.5.2. Using Comparable

Lab9.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Customer implements Comparable<Customer>{
    int cid;
    String cname;
    long phone;

    public Customer() {}
    public Customer(int cid, String cname, long phone) {
        super();
        this.cid = cid;
        this.cname = cname;
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "[" + cid + ", " + cname + ", " + phone + "]";
    }

    @Override
    public int compareTo(Customer cust) {
        return this.cid-cust.cid;
    }
}
```



```
public class Lab9{

    public static void main(String[] args) {

        Customer cust1 = new Customer(102,"sd",222);
        Customer cust2 = new Customer(103,"ds",333);
        Customer cust3 = new Customer(101,"sri",111);

        List<Customer> customers = new ArrayList<>();
        customers.add(cust1);
        customers.add(cust2);
        customers.add(cust3);

        Collections.sort(customers);

        for(Customer cust:customers) {
            System.out.println(cust);
        }

    }
}
```



9.5.3. Using Comparator

Lab10.java

```
package com.jlcindia.collections.sort;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyMarksComparator implements Comparator<Student> {

    @Override
    public int compare(Student stu1, Student stu2) {
        return (int) (stu1.marks - stu2.marks);
    }
}

class Student implements Comparable<Student> {

    int sid;
    String sname;
    double marks;

    public Student() { }
    public Student(int sid, String sname, double marks) {
        super();
        this.sid = sid;
        this.sname = sname;
        this.marks = marks;
    }
}
```



```
@Override
public String toString() {
    return "[" + sid + ", " + sname + ", " + marks + "]";
}

@Override
public int compareTo(Student stu) {
    return this.sid - stu.sid;
}
}
public class Lab10 {
    public static void main(String[] args) {

        Student stu1 = new Student(102, "sd", 70);
        Student stu2 = new Student(103, "ds", 60);
        Student stu3 = new Student(101, "sri", 50);

        List<Student> students = new ArrayList<>();
        students.add(stu1);
        students.add(stu2);
        students.add(stu3);

        Collections.sort(students);

        for (Student stu : students) {
            System.out.println(stu);
        }
        System.out.println("-----");

        Collections.sort(students, new MyMarksComparator());

        for (Student stu : students) {
            System.out.println(stu);
        }
    }
}
```



9.6. Exploring Bubble Sort

- ♦ Bubble sort is the simplest sorting algorithm.
- ♦ It works by iterating the input array from the first element to the last, comparing each pair of elements and swapping them if needed.
- ♦ **The only significant advantage that bubble sort has over other implementations is that it can detect whether the input list is already sorted or not.**

Complexity of Bubble Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Worst	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Space Complexity	$O(1)$

9.6.1. Bubble Sort Implementation

Lab11.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab11 {
    public static void bubbleSort(int arr[]) {

        int n= arr.length;
```



```
        for(int i=0;i<n;i++) {
            for(int j=0;j<n-i-1;j++) {
                if(arr[j]>arr[j+1]) {
                    int temp = arr[j];
                    arr[j]=arr[j+1];
                    arr[j+1]=temp;
                }
            }
        }

        public static void main(String[] args) {
            int arr[] = {5,9,15,8,20,7,10};
            System.out.println(Arrays.toString(arr));

            bubbleSort(arr);
            System.out.println(Arrays.toString(arr));
        }
    }
```

9.6.2. Optimized Bubble Sort

Lab12.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab12 {
    public static void bubbleSort(int arr[]) {

        int n= arr.length;
```



```
        for(int i=0;i<n;i++) {  
            boolean swaped=false;  
            for(int j=0;j<n-i-1;j++) {  
                if(arr[j]>arr[j+1]) {  
                    int temp = arr[j];  
                    arr[j]=arr[j+1];  
                    arr[j+1]=temp;  
                    swaped=true;  
                }  
            }  
            if(swaped==false) {  
                break;  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
        int arr[] = {5,9,15,8,20,7,10};  
        System.out.println(Arrays.toString(arr));  
        bubbleSort(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```



9.7. Exploring Selection Sort

- ♦ Easy to implement the Selection sort
- ♦ Selection sort works well for small data sets.
- ♦ Selection sort is an in-place sorting algorithm.
- ♦ Selection sort is not a Stable sorting algorithm.

Algorithm

- 1) Select the smallest element in the list
- 2) Swap that with the element in the current position
- 3) Repeat this process for all the elements until the entire array is sorted

- ♦ This algorithm is called selection sort since it repeatedly selects the smallest element.

Complexity of Selection Sort:

Time Complexity - Best	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

9.7.1. Selection Sort Implementation

Lab13.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab13 {  
  
    public static void selectionSort(int arr[]) {  
  
        int n = arr.length;  
  
        for (int i = 0; i < n-1; i++) {  
            int min_index = i;  
            for (int j = i + 1; j < n; j++) {  
                if (arr[j] < arr[min_index])  
                    min_index = j;  
            }  
  
            if(i!=min_index){  
                int temp = arr[min_index];  
                arr[min_index] = arr[i];  
                arr[i] = temp;  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
  
        int arr[] = { 10, 5, 30, 25, 15, 20 };  
  
        System.out.println(Arrays.toString(arr));  
        selectionSort(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```



9.8. Exploring Insertion Sort

- ♦ Insertion sort is a simple and efficient comparison sort.
- ♦ Insertion sort is an in-place sorting algorithm.
- ♦ Insertion sort is a Stable sorting algorithm.

Algorithm

- 1) Consider some part is already Sorted
 - 2) Take i^{th} element and Insert it in the Correct position
 - 3) Repeat this process for all the elements until the entire array is sorted
- ♦ This algorithm is called Insertion Sort since it repeatedly Inserts the Element in Correct position in the Sorted part.

Complexity of Insertion Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

9.8.1. Insertion Sort Implementation

Lab14.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab14 {
```



```
public static void insertionSort(int arr[]) {
```

```
    int n = arr.length;
```

```
    for (int i = 1; i < n; i++) {
```

```
        int key = arr[i];
```

```
        int j=i-1;
```

```
        while(j>=0 && arr[j]>key) {
```

```
            arr[j+1] = arr[j];
```

```
            j--;
```

```
        }
```

```
        arr[j+1] = key;
```

```
    }
```

```
}
```

```
public static void main(String[] args) {
```

```
    int arr[] = { 10, 5, 30, 25, 15, 20 };
```

```
    System.out.println(Arrays.toString(arr));
```

```
    insertionSort(arr);
```

```
    System.out.println(Arrays.toString(arr));
```

```
}
```

```
}
```



9.9. Exploring Merge Sort

- ♦ Merge sort is an example of the divide and conquer strategy.
- ♦ Merging is the process of combining two sorted arrays into one bigger sorted array
- ♦ Merge sort divides the list into two parts; then each part is conquered individually.
- ♦ Merge sort is not an in-place sorting algorithm.
- ♦ Merge sort is a Stable sorting algorithm.
- ♦ This algorithm is used for sorting a linked list

Algorithm

- 1) Divide the Input Array or List into two parts.
 - 2) Solve each part recursively.
 - 3) After solving the sub problems, they are merged by scanning the resultant sub problems.
- ♦ This algorithm is called Merge Sort since it is merging two sorted arrays to make one bigger sorted array.

Complexity of Merge Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n \log n)$
Space Complexity	$O(n)$



9.9.1. Merge two Sorted Arrays

Lab15A.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab15A {
    public static void mergeArrays(int a[], int b[]) {
        int m = a.length;
        int n = b.length;

        int i=0;
        int j=0;
        while(i<m && j<n) {

            if(a[i]<=b[j]) {
                System.out.println(a[i]);
                i++;
            }else {
                System.out.println(b[j]);
                j++;
            }
        }
        while(i<m) {
            System.out.println(a[i]);
            i++;
        }
        while(j<n) {
            System.out.println(b[j]);
            j++;
        }
    }
}
```



```
public static void main(String[] args) {  
    int a[] = { 10, 20, 50, 60, 70 };  
    int b[] = { 5, 15, 50 };  
  
    mergeArrays(a, b);  
}  
}
```

Lab15B.java

```
package com.jlccindia.sorting;  
  
import java.util.Arrays;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public class Lab15B {  
    public static int [] mergeArrays(int a[], int b[]) {  
        int m = a.length;  
        int n = b.length;  
  
        int temp[] = new int[m+n];  
  
        int i=0;  
        int j=0;  
        int k=0;  
        while(i<m && j<n) {  
  
            if(a[i]<=b[j]) {  
                temp[k++] = a[i];  
                i++;  
            }else {  
                temp[k++] = b[j];  
                j++;  
            }  
        }  
    }  
}
```



```
        while(i<m) {
            temp[k++] = a[i];
            i++;
        }

        while(j<n) {
            temp[k++] = b[j];
            j++;
        }

        return temp;
    }

    public static void main(String[] args) {
        int a[] = { 10, 20, 50, 60, 70 };
        int b[] = { 5, 15, 50 };

        int c [] = mergeArrays(a, b);
        System.out.println(Arrays.toString(c));
    }
}
```



9.9.2. Implement Merge Function

Lab16.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab16 {
    public static void merge(int arr[], int low,int mid,int high) {

        int m = mid-low;
        int n = high-mid+1;

        int a[] = new int[m];
        for(int i=0;i<m;i++) {
            a[i] = arr[i+low];
        }

        int b[] = new int[n];
        for(int j=0;j<n;j++) {
            b[j] = arr[j+mid];
        }

        int i=0;
        int j=0;
        int k=low;
        while(i<m && j<n) {
            if(a[i]<=b[j]) {
                arr[k++] = a[i];
                i++;
            }else {
                arr[k++] = b[j];
                j++;
            }
        }
    }
}
```



```
    }

    while(i<m) {
        arr[k++] = a[i];
        i++;
    }

    while(j<n) {
        arr[k++] = b[j];
        j++;
    }

}

public static void main(String[] args) {
    int arr[] = { 10, 20, 50, 60, 70, 5, 15, 50 };
    int low = 0;
    int high = arr.length-1;
    int mid = 5;

    System.out.println(Arrays.toString(arr));
    merge(arr, low, mid, high);
    System.out.println(Arrays.toString(arr));

}
}
```



9.9.3. Merge Sort Algorithm

Lab17.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab17 {
    public static void merge(int arr[], int low, int mid, int high) {

        int m = mid - low + 1;
        int n = high - mid;

        int a[] = new int[m];
        for (int i = 0; i < m; i++) {
            a[i] = arr[i + low]; // IMP
        }

        int b[] = new int[n];
        for (int j = 0; j < n; j++) {
            b[j] = arr[j + mid + 1]; // IMP
        }

        int i = 0;
        int j = 0;
        int k = low; // IMP
        while (i < m && j < n) {
            if (a[i] <= b[j]) {
                arr[k++] = a[i];
                i++;
            } else {
                arr[k++] = b[j];
                j++;
            }
        }
    }
}
```



```
    }

    while (i < m) {
        arr[k++] = a[i];
        i++;
    }

    while (j < n) {
        arr[k++] = b[j];
        j++;
    }

}

public static void mergeSort(int arr[], int low, int high) {

    if (low < high) {
        int mid = low + (high - low) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }

}

public static void main(String[] args) {
    int arr[] = { 10, 5, 50, 30, 15, 7, 25 };
    int low = 0;
    int high = arr.length - 1;

    System.out.println(Arrays.toString(arr));
    mergeSort(arr, low, high);
    System.out.println(Arrays.toString(arr));
}

}
```



9.10. Exploring Quicksort

- ♦ Quick sort is an example of the divide and conquer strategy.
- ♦ It is one of the famous algorithms among comparison-based sorting algorithms
- ♦ It is also called partition exchange sort.
- ♦ It uses recursive calls for sorting the elements,
- ♦ Quick sort is an in-place sorting algorithm.
- ♦ Quick sort is a Non-Stable sorting algorithm.

Algorithm

- ♦ **The recursive algorithm consists of 3 steps:**
 - 1) Pick an element in the array to serve as the “pivot” point.
 - 2) Split the array into two parts –
 - a. First Part with elements smaller than the pivot
 - b. Second Part with elements larger than the pivot.
 - 3) Recursively repeat the algorithm for both halves of the original array.

Complexity of Quick Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(n)$

9.10.1. Partitioning the given Array

Lab18.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab18 {  
    //Time Complexity - O(3n)  
    //Space Complexity - O(n)  
    public static void simplePartition(int arr[], int low, int high) {  
  
        int pivot = arr[high]; //8  
        int n = high-low+1;  
  
        int temp[] = new int[n];  
        int k = low;  
  
        for(int i=low;i<=high;i++) {  
            if(arr[i]<=pivot)  
                temp[k++] = arr[i];  
        }  
  
        for(int i=low;i<=high;i++) {  
            if(arr[i]>pivot)  
                temp[k++] = arr[i];  
        }  
  
        for(int i=low;i<=high;i++) {  
            arr[i]= temp[i-low] ;  
        }  
    }  
  
    public static void main(String[] args) {  
        int arr[] = { 7,5,13,6,9,12,4,8 };  
        int low = 0;  
        int high = arr.length - 1;  
  
        System.out.println(Arrays.toString(arr));  
        simplePartition(arr, low, high);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```



9.10.2. Lomuto Partition

Lab19.java

```
package com.jlcindia.sorting;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab19 {
    //Time Complexity - O(n)
    //Space Complexity - O(1)
    public static void lomutoPartition(int arr[], int low, int high) {

        int pivot = arr[high]; //8
        int i=low-1;

        for(int j=low;j<=high;j++) {
            if(arr[j]<=pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        public static void main(String[] args) {
            int arr[] = { 7,5,13,6,9,12,4,8 };
            int low = 0;
            int high = arr.length - 1;

            System.out.println(Arrays.toString(arr));
            lomutoPartition(arr, low, high);
            System.out.println(Arrays.toString(arr));
        }
    }
}
```



9.10.3. Hoares Partition

Lab20A.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab20A {
    // Time Complexity - O(n)
    // Space Complexity - O(1)
    public static int hoaresPartition(int arr[], int low, int high) {
        int pivot = arr[low];
        int i = low;
        int j = high ;

        while (true) {
            // 1. i Task
            while (arr[i] < pivot) {
                i++;
            }
            // 2. j Task
            while (arr[j] > pivot) {
                j--;
            }
            // 3. Return
            if (i >= j)
                return j;

            // 4.swap(i,j)
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
```



```
public static void main(String[] args) {
    int arr[] = { 6, 5, 13, 7, 9, 12, 4, 8 };
    int low = 0;
    int high = arr.length - 1;

    System.out.println(Arrays.toString(arr));
    int j = hoaresPartition(arr, low, high);
    System.out.println(Arrays.toString(arr));
    System.out.println("j = "+j);
    System.out.println("arr[j] = "+arr[j]);
}
}
```

Lab20B.java

```
package com.jlcindia.sorting;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab20B {

    // Time Complexity - O(n)
    // Space Complexity - O(1)
    public static int hoaresPartition(int arr[], int low, int high) {

        int pivot = arr[low]; // 7
        int i = low - 1;
        int j = high + 1;
```

```
        while (true) {
            // 1. i Task
            do {
                i++;
            } while (arr[i] < pivot);

            // 2. j Task
            do {
                j--;
            } while (arr[j] > pivot);

            // 3. Return
            if (i >= j)
                return j;

            // 4.swap(i,j)
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    public static void main(String[] args) {
        int arr[] = { 7, 5, 13, 6, 9, 12, 4, 8 };
        int low = 0;
        int high = arr.length - 1;

        System.out.println(Arrays.toString(arr));
        hoaresPartition(arr, low, high);
        System.out.println(Arrays.toString(arr));
    }
}
```



9.10.4. Quick Sort with Lomuto Partition

Lab21.java

```
package com.jlcindia.sorting;

import java.util.Arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab21 {

    public static int lomutoPartition(int arr[], int low, int high) {

        int pivot = arr[high];
        int i=low-1;

        for(int j=low;j<=high-1;j++) {
            if(arr[j]<pivot) {
                i++;
                int temp= arr[i];
                arr[i]=arr[j];
                arr[j]=temp;
            }
        }

        int temp= arr[i+1];
        arr[i+1]=arr[high];
        arr[high]=temp;

        return (i+1);
    }
}
```



```
public static void quickSort(int arr[], int low, int high) {
    if(low<high) {
        int pivotIndex=lomutoPartition(arr, low, high);
        quickSort(arr, low, pivotIndex-1);
        quickSort(arr, pivotIndex+1, high);
    }
}

public static void main(String[] args) {
    int arr[] = {8,4,7,9,3,10,5};
    int low = 0;
    int high = arr.length-1;
    System.out.println(Arrays.toString(arr));
    quickSort(arr,low,high);
    System.out.println(Arrays.toString(arr));
}
}
```



9.10.5. Quick Sort with Hoares Partition

Lab22.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab22 {
    public static int hoaresPartition(int arr[], int low, int high) {

        int pivot = arr[low];
        int i = low - 1;
        int j = high + 1;

        while (true) {
            do {
                i++;
            } while (arr[i] < pivot);

            do {
                j--;
            } while (arr[j] > pivot);

            if (i >= j) {
                return j;
            }

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
```



```
public static void quickSort(int arr[], int low, int high) {  
  
    if(low<high) {  
        int pivotIndex=hoaresPartition(arr, low, high);  
        quickSort(arr, low, pivotIndex);  
        quickSort(arr, pivotIndex+1, high);  
    }  
  
}  
  
public static void main(String[] args) {  
  
    int arr[] = {8,4,7,9,3,10,5};  
    int low = 0;  
    int high = arr.length-1;  
  
    System.out.println(Arrays.toString(arr));  
    quickSort(arr,low,high);  
    System.out.println(Arrays.toString(arr));  
  
}  
}
```



9.11. Exploring Cycle Sort

- ♦ Cycle Sort performs the Sorting with minimum memory writes and can be useful for cases where the memory writes are costly.
- ♦ Selection sort is an in-place sorting algorithm.
- ♦ Selection sort is not a Stable sorting algorithm.

Complexity of Cycle Sort:

Time Complexity - Best	$O(n^2)$
Time Complexity - Average	$O(n^2)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(1)$

9.11.1. Cycle Sort with Distinct elements

Lab23.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab23 {

    public static void cycleSort(int arr[]) {

        int n = arr.length;
        for (int cycle = 0; cycle < n - 1; cycle++) {
            int item = arr[cycle];
            int position = cycle;
            for (int j = cycle + 1; j < n; j++) {
                if (arr[j] < item)
                    position++;
            }
        }
    }
}
```



```
        int temp = item;
        item = arr[position];
        arr[position] = temp;

        while (position != cycle) {
            position = cycle;
            for (int j = cycle + 1; j < n; j++) {
                if (arr[j] < item)
                    position++;
            }

            temp = item;
            item = arr[position];
            arr[position] = temp;
        }
    }

    public static void main(String[] args) {

        int arr[] = { 20,40,50,10,30 };

        System.out.println(Arrays.toString(arr));
        cycleSort(arr);
        System.out.println(Arrays.toString(arr));

    }
}
```



9.12. Exploring Heap Sort

- ♦ Basic Idea of Heap Sort is based on Selection Sort
- ♦ Heap Sort is an Improvement of Selection Sort

In Selection Sort

- a) We find the Max Element
- b) We swap Max Element with Last element

- ♦ In Selection Sort, we do Linear Search to find the max element.
- ♦ That why Time Complexity with Selection Sort is $O(n^2)$

In Heap Sort

- a) We build Max Heap
- b) We swap Root with Last element - So the Max Element goes to End.
- c) Reduce the Heap Size
- d) Heapify the Array
- e) Repeat Steps b,c,d

- ♦ Heap Sort does the Optimization over the selection sort. Uses the same Idea but for finding the Max Element, Uses Max Heap Data Structures instead of Linear Search
- ♦ That why Time Complexity with Heap Sort is $O(n \log n)$

Complexity of Heap Sort:

Time Complexity - Best	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$
Time Complexity - Worst	$O(n \log n)$
Space Complexity	$O(1)$



9.12.1. Heap Sort Implementation

Lab24.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab24 {
    public static void heapify(int arr[], int n, int i) {

        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            // Recursively heapify the affected sub-tree
            heapify(arr, n, largest);
        }
    }
}
```



```
public static void buildHeap(int arr[]) {

    int n = arr.length;
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

public static void heapSort(int arr[]) {

    int n = arr.length;
    buildHeap(arr);
    for (int i = n - 1; i >= 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

public static void main(String[] args) {

    int arr[] = { 10, 15, 50, 5, 20, 60, 2 };

    System.out.println(Arrays.toString(arr));
    heapSort(arr);
    System.out.println(Arrays.toString(arr));
}
}
```